

Smart Event & Volunteer Marketplace

A Project Report submitted in partial fulfilment of the
Requirements for the award of the degree of

**Bachelor of Technology
In**

Computer Science and Engineering

By

Pula Vivek (24015A0503)

T. Sri Vatchava (24015A0504)

Mohammed Jamaluddin (24015A0511)

Under The Guidance of

Ms. M.N. Shahenaaz Sultana

Assistant Professor

Department of CSE



**Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science & Technology
Hyderabad**

**Kukatpally, Hyderabad - 500 085.
May 2026**

**Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science & Technology
Hyderabad
Kukatpally, Hyderabad - 500 085.**



DECLARATION BY THE CANDIDATES

We, Pula Vivek (24015A0503), Thatikonda Sri Vatchava (24015A0504), and Mohammed Jamaluddin (24015A0511), hereby declare that the project report entitled “**SMART EVENT AND VOLUNTEER MARKETPLACE**”, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by us under the guidance of **Ms. M.N. Shahenaaz Sultana**.

We further declare that this project work has been completed by us independently and has not been submitted previously, either wholly or partially, to any other University or Institution for the award of any degree or diploma. Wherever materials, data, references, or ideas from other sources have been used, proper acknowledgment and due credit have been given in the report.

Pula Vivek (24015A0503)

Thatikonda Sri Vatchava (24015A0504)

Mohammed Jamaluddin (24015A0511)

Date:

**Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science & Technology
Hyderabad**

**Kukatpally, Hyderabad - 500 085.
May 2026**



CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled “**SMART EVENT AND VOLUNTEER MARKETPLACE**”, being submitted by **PULA VIVEK (24015A0503), THATIKONDA SRI VATCHAVA (24015A0504), and MOHAMMED JAMALUDDIN (24015A0511)**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them.

The results of investigation enclosed in this report have been verified and found satisfactory. The results embodied in the project have not been submitted to any other University or Institute for the award of any other Degree or Diploma.

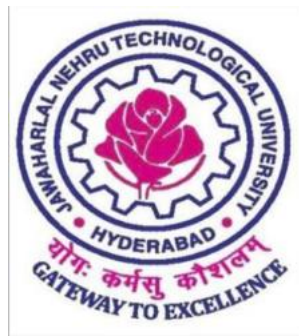
Ms. M.N. Shahenaaz Sultana
Assistant Professor

Date:

**Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science & Technology
Hyderabad**

Kukatpally, Hyderabad - 500 085.

May 2026



CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project report entitled “**SMART EVENT AND VOLUNTEER MARKETPLACE**”, being submitted by **PULA VIVEK (24015A0503), THATIKONDA SRI VATCHAVA (24015A0504), and MOHAMMED JAMALUDDIN (24015A0511)**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them.

Head Of the Department

Dr. K Santhi Sree

Professor and Head of the Department

Date:

ABSTRACT

Smart Event & Volunteer Marketplace is a full-stack web platform developed to simplify and modernize event management by integrating event discovery, participant registration, volunteer coordination, and organizer operations into a single, user-friendly system. The platform addresses common challenges faced during event execution such as manual volunteer handling, unverified attendance, registration overload and fragmented communication workflows by providing a structured and centralized solution for both organizers and participants.

The platform includes an event discovery system that allows users to browse and filter events based on category, date, location, and availability. Users can register for both free and paid events, with secure online payment processing handled through Razorpay integration. A volunteer management module enables organizers to define volunteer requirements, manage applications, participants, and track volunteer capacity in real time. The system also introduces QR-based check-in verification for attendees and volunteers, ensuring authentic participation records and reducing manual verification efforts during event execution.

A centralized organizer dashboard provides real-time monitoring of registrations, volunteer status, attendance analytics, seat availability, and waitlist management. Organizers can track event performance, manage participant workflows, and oversee operational activities through an interactive interface designed for efficient event administration. The platform also supports automated capacity management to prevent overbooking and maintain accurate participant limits during high-demand registrations.

The application is built using the MERN stack, with React.js powering the frontend interface, Node.js and Express.js handling backend APIs and business logic, and MongoDB serving as the primary database for scalable data management. Additional integrations such as Razorpay API and QR-based verification systems enhance security, reliability, and operational efficiency throughout the platform.

By consolidating event operations, volunteer coordination, registration workflows, and attendance verification into one cohesive ecosystem, the app improves event organization efficiency, enhances participant experience, and provides organizers with data-driven operational control. The platform is currently under active development and is designed to support real-world deployment for college fests, technical events, workshops, and community-driven programs.

ACKNOWLEDGEMENTS

We would like to express our heartfelt gratitude to everyone who supported and contributed to the successful development of the Smart Event & Volunteer Marketplace project.

First and foremost, we extend our sincere thanks to our respected mentor, Ms. M.N. Shahenaaz Sulthana Garu, for her valuable guidance, continuous support, and constructive suggestions throughout the project. Her encouragement and mentorship helped us stay focused and complete the project effectively.

We are also thankful to the global community of developers and open-source contributors whose technologies—particularly React.js, Node.js, Express.js, MongoDB, and Razorpay—played an important role in the development of our application. Their tools, documentation, and community resources greatly supported our learning and implementation process.

Our appreciation also goes to the faculty members and classmates, for their support, suggestions, and motivation during the development of the project. Their feedback helped us improve the functionality and usability of the platform.

Finally, we would like to thank our families and friends for their constant encouragement and support throughout this journey. Their motivation helped us complete the project with confidence and dedication.

Pula Vivek (24015A0503)

Thatikonda Sri Vatchava (24015A0504)

Mohammed Jamaluddin (24015A0511)

TABLE OF CONTENTS

ABSTRACT.....	5
1.Introduction	10
1.1 Problem Statement.....	11
Problems	12
1.2 Purpose.....	12
1.3 Objectives.....	13
The specific objectives	14
1.4 Scope	15
2. Requirement Analysis	16
2.1 Hardware Requirements	16
2.1.1 Server Requirements.....	16
2.1.2 Client Requirements	16
2.2 Software Requirements.....	16
2.2.1 Frontend Software.....	17
2.2.2 Backend Software	17
2.3 Functional Requirements	18
2.4 Non- Functional Requirements	18
3. UML DIAGRAMS.....	19
3.1 CLASS DIAGRAM	19
3.2 USE CASE DIAGRAM	20
3.3 SEQUENCE DIAGRAM	21
3.4 SCHEMA	22
3.5 Data flow diagram.....	23
4. PROJECT DESIGN AND DEVELOPMENT	24
4.1Frontend Design	24
4.2 Backend Design.....	24
4.3 Database Design.....	25
4.4 Step-by-Step Development Workflow	26
5. TOOLS AND TECHNOLOGIES USED	29
5.1 Backend Technologies	29
5.2 Frontend Technologies.....	29
6.Project Phases	31
1. Initiation	31

2. Planning	31
3. Development	31
4. Integration	32
5. Optiimization	32
6. Deployment	32
7. Testing.....	33
7.1 Unit Testing.....	33
7.2 Integration Testing	33
7.3 Functional Testing	34
7.4 Usability Testing	34
7.5 Performance Testing.....	34
7.6 Regression Testing.....	35
7.7 User Acceptance Testing (UAT)	36
8. USER GUIDE.....	37
8.1 Authentication	37
8.2 Home/Event discovery page	38
8.3 Event page	38
8.4 Profile	39
8.5 Organizer Dasshboard.....	40
8.6 Payments	40
8.7 QR verification.....	41
8.8 Volunteer Workflow	43
9. Conclusion.....	44
10.References	45

1. INTRODUCTION

The Smart Event & Volunteer Marketplace is a comprehensive MERN-stack based platform designed to modernize the way events are organized, managed, and executed within educational institutions, technical communities, and organizations. Traditional event management systems primarily focus on event listing and ticket booking while ignoring operational aspects such as volunteer coordination, attendance verification, participant engagement, and real-time analytics. This project addresses those gaps by integrating event discovery, registration workflows, volunteer management, QR-based verification, and organizer dashboards into a single unified ecosystem.

In colleges and universities, event management is often handled manually using Google Forms, WhatsApp groups, spreadsheets, and offline coordination. Such approaches create operational inefficiencies including duplicate registrations, lack of attendee verification, communication delays, and difficulty in tracking volunteer contributions. Event organizers also struggle with handling waitlists, monitoring event performance, and maintaining accurate participation records. These limitations become more severe during large-scale technical fests, workshops, hackathons, and cultural events where thousands of registrations must be handled reliably.

The proposed platform introduces a centralized digital solution capable of supporting both free and paid events with secure online transactions using Razorpay integration. Participants can discover events using filters such as category, date, and availability, while volunteers can apply for operational roles based on skills and organizer requirements. Organizers gain access to real-time dashboards that display attendee metrics, volunteer status, event performance, and operational analytics.

The system also implements QR-based attendance verification for both attendees and volunteers. Each registration generates a unique QR code which is scanned during event check-in. This ensures accurate participation tracking, eliminates fake

attendance claims, and automates event verification processes.

The platform is built using React.js for the frontend interface, Node.js and Express.js for backend APIs and business logic, and MongoDB for scalable database management. Razorpay is integrated for secure payment processing, while JWT authentication ensures secure access control for users, organizers, and administrators.

The overall objective of the project is to create a scalable, secure, and user-friendly event ecosystem that simplifies event operations while improving participant experience and organizer efficiency.

1.1 Problem Statement

Current In many colleges and organizations, event management is still handled using fragmented and manual methods such as Google Forms, spreadsheets, WhatsApp groups, and physical attendance sheets. While existing event platforms support basic event listing and registration features, they often fail to provide proper tools for volunteer coordination, attendance verification, real-time seat management, and organizer monitoring. As a result, managing large-scale events becomes difficult and time-consuming for organizers.

Another major limitation of current systems is the lack of centralized operational control. Organizers face challenges in tracking participant registrations, managing volunteer responsibilities, verifying attendance, and monitoring event performance. Manual coordination frequently leads to communication gaps, duplicate registrations, overbooking issues, and inaccurate attendance records. These problems highlight the need for a structured and reliable platform that can simplify event execution, automate operational workflows, and improve overall event management efficiency.

Problems Faced:

- Lack of proper volunteer management and coordination
- Manual attendance tracking and proxy participation issues
- Difficulty managing real-time seat availability and waitlists
- Duplicate registrations and booking inconsistencies
- Limited support for secure online payment processing
- Lack of centralized organizer dashboards and monitoring tools
- Poor communication between organizers, volunteers, and participants
- Absence of post-event analytics and operational insights
- Increased manual workload during event execution
- Difficulty handling large-scale events efficiently

1.2 Purpose of the Project

The purpose of the Smart Event & Volunteer Marketplace is to provide an integrated digital ecosystem that simplifies event operations for organizers while improving accessibility and experience for participants and volunteers.

The project aims to eliminate fragmented workflows currently dependent on multiple third-party tools such as spreadsheets, messaging platforms, and manual attendance systems. By consolidating event management functionalities into a unified platform, the system improves operational transparency, reduces manual workload, and ensures reliable participation tracking.

For participants, the platform offers a seamless event discovery and registration experience. Users can browse events, view availability, complete registrations, and receive confirmation instantly. Paid registrations are handled securely through Razorpay payment integration.

For volunteers, the platform creates a structured environment where users can apply for roles based on skill requirements, availability, and events. Organizers can

review applications, approve volunteers, and monitor participation effectively.

For organizers, the platform provides centralized control over event operations including attendee management, volunteer coordination, QR-based verification, waitlist automation, and analytics reporting. Real-time dashboards improve operational decision-making and event execution efficiency.

The system is also designed with scalability and modularity in mind. Additional modules such as mobile applications, AI recommendations, certificate generation, and institution-level integrations can be added in future updates.

1.3 Objectives

The primary objective of the project is to design and develop a scalable and efficient event and volunteer management platform using modern web technologies. The system aims to simplify event operations by integrating event registration, volunteer coordination, attendance verification, payment handling, and organizer monitoring into a centralized platform. The project also focuses on improving participant experience, reducing manual workload, and enabling organizers to manage events more effectively through automation and real-time operational tools.

The specific Objectives are:

1. Centralized Event Management

Develop a unified platform where users can discover, browse, and register for events easily from a single system.

2. Secure Registration and Payments

Support both free and paid event registrations using secure Razorpay payment integration for reliable transaction processing.

3. Volunteer Coordination System

Implement a structured workflow for volunteer applications, approvals, role assignment, and participation tracking.

4. QR-Based Attendance Verification

Provide QR code-based check-in functionality to ensure verified attendance for participants and volunteers.

5. Real-Time Capacity Management

Enable live tracking of event seat availability and automate waitlist handling to prevent overbooking.

6. Organizer Dashboard and Analytics

Develop dashboards that allow organizers to monitor registrations, volunteer activity, attendance status, and event performance.

7. Secure Authentication and Authorization

Implement role-based authentication using JWT to ensure secure access control for users and organizers.

8. Responsive User Interface

Design a user-friendly and responsive frontend using React.js for smooth accessibility across different devices.

9. Scalable Database Architecture

Use MongoDB to maintain flexible and scalable data management for users, events, registrations, and payments.

10. Automation and Operational Efficiency

Reduce manual event management tasks by automating registration workflows, attendance verification, and participant monitoring.

1.4 Scope of the Project

The scope of the Smart Event & Volunteer Marketplace covers multiple operational and participant-focused functionalities related to event execution and volunteer management.

The system supports:

- Event creation and management
- Event discovery and filtering
- Free and paid registrations
- Razorpay payment integration
- Volunteer application and approval workflows
- QR code generation and scanning
- Real-time seat availability tracking
- Organizer analytics dashboards
- Waitlist automation
- User authentication and authorization

The platform is intended for use in educational institutions, technical communities, startup ecosystems, workshops, hackathons, cultural festivals, and conferences.

The project does not currently include:

- AI-based recommendation systems
- Mobile application deployment
- Institution ERP integrations
- Automated certificate generation
- Multi-language support

However, the architecture is modular and scalable, allowing these features to be added in future versions.

2. REQUIREMENT ANALYSIS

Requirement analysis plays a critical role in defining the functional and non-functional requirements of the proposed system. It ensures that the system architecture, workflows, and technologies align with operational objectives and user expectations.

The Smart Event & Volunteer Marketplace requires reliable frontend technologies, scalable backend services, secure authentication mechanisms, and efficient database management to support concurrent users and real-time event operations.

2.1 Hardware Requirements

2.1.1 Server Requirements:

- **Processor:** Intel Xeon or equivalent cloud-based infrastructure
- **RAM:** Minimum 8 GB
- **Storage:** SSD-based storage for fast database access
- **Internet Connectivity:** High-speed broadband
- **Cloud Hosting:** Render, Railway, or AWS deployment support

2.1.2 Client Requirements:

- **Devices:** Laptop, desktop, smartphone, or tablet
- **RAM:** Minimum 4 GB
- **Browser Support:** Chrome, Firefox, Edge
- **Internet Connectivity:** Stable internet connection for real-time updates

2.2 Software Requirements

2.2.1 Frontend Software:

- **React.js:**
Used for building the interactive and responsive frontend interface of the application.

- **Tailwind CSS / Bootstrap:**

Used for designing responsive layouts and improving UI styling consistency.

- **Axios:**

Used for sending API requests and handling communication between frontend and backend.

2.2.2 Backend Software:

- **Node.js:**

Provides the runtime environment for executing backend server operations.

- **Express.js:**

Used for creating REST APIs and managing backend routing and middleware.

- **JWT Authentication:**

Implements secure login sessions and role-based user authorization.

- **Razorpay API:**

Used for secure online payment processing and transaction verification.

2.2.3 Database

- **MongoDB with Mongoose ODM:**

Used for storing and managing application data such as users, events, registrations, and payments.

2.2.4 Development Tools

- **VS Code:**
Used as the primary source-code editor for development.
- **Postman:**
Used for API testing and debugging.
- **Git & GitHub:**
Used for version control and collaborative development.

2.2.5 Deployment Platforms

- **Vercel:**
Used for deploying and hosting the frontend application.
- **Render / Railway:**
Used for backend hosting and server deployment.

2.3 Functional Requirements

1. User Authentication:

Users must be able to register, login, and manage profiles securely.

2. Event Discovery:

Users should browse events using search filters and categories.

3. Registration System:

Participants should register for free or paid events.

4. Volunteer Management:

Volunteers should apply for roles and receive approval status.

5. QR Verification:

The system should generate QR codes for attendance verification.

6. Organizer Dashboard:

Organizers should manage registrations, volunteers, and analytics.

7. Payment Handling:

Secure online payments should be processed using Razorpay.

8. Capacity Management:

The system should prevent overbooking and manage waitlists automatically.

2.4 Non-Functional Requirements

1. Scalability:

The platform should support large numbers of concurrent users.

2. Security:

Sensitive data such as passwords and payment details must be protected.

3. Reliability:

The system should maintain high uptime and consistent performance.

4. Performance:

API responses and page loading should remain efficient under load.

5. Usability:

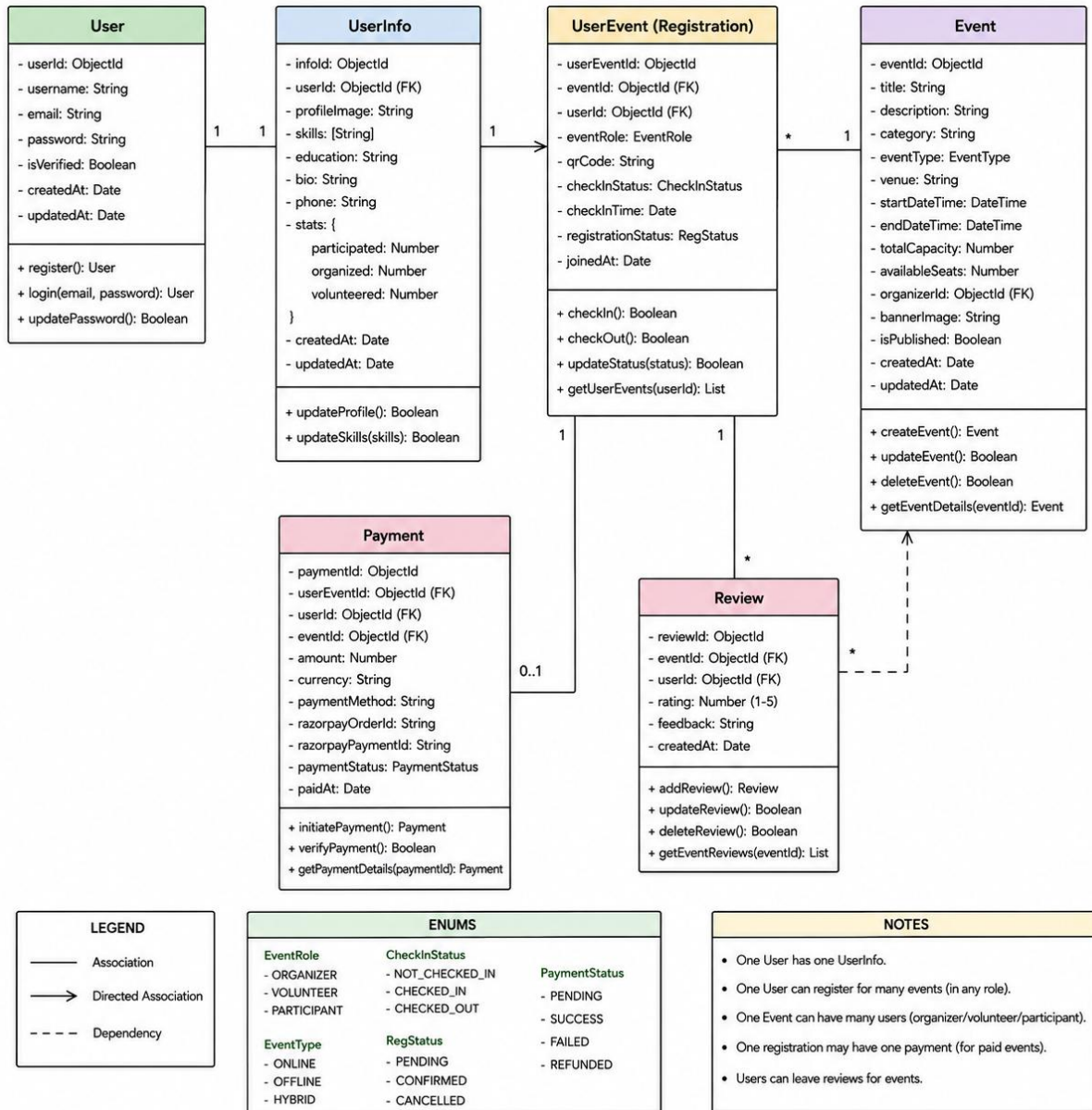
The user interface should remain intuitive and responsive across devices.

6. Maintainability:

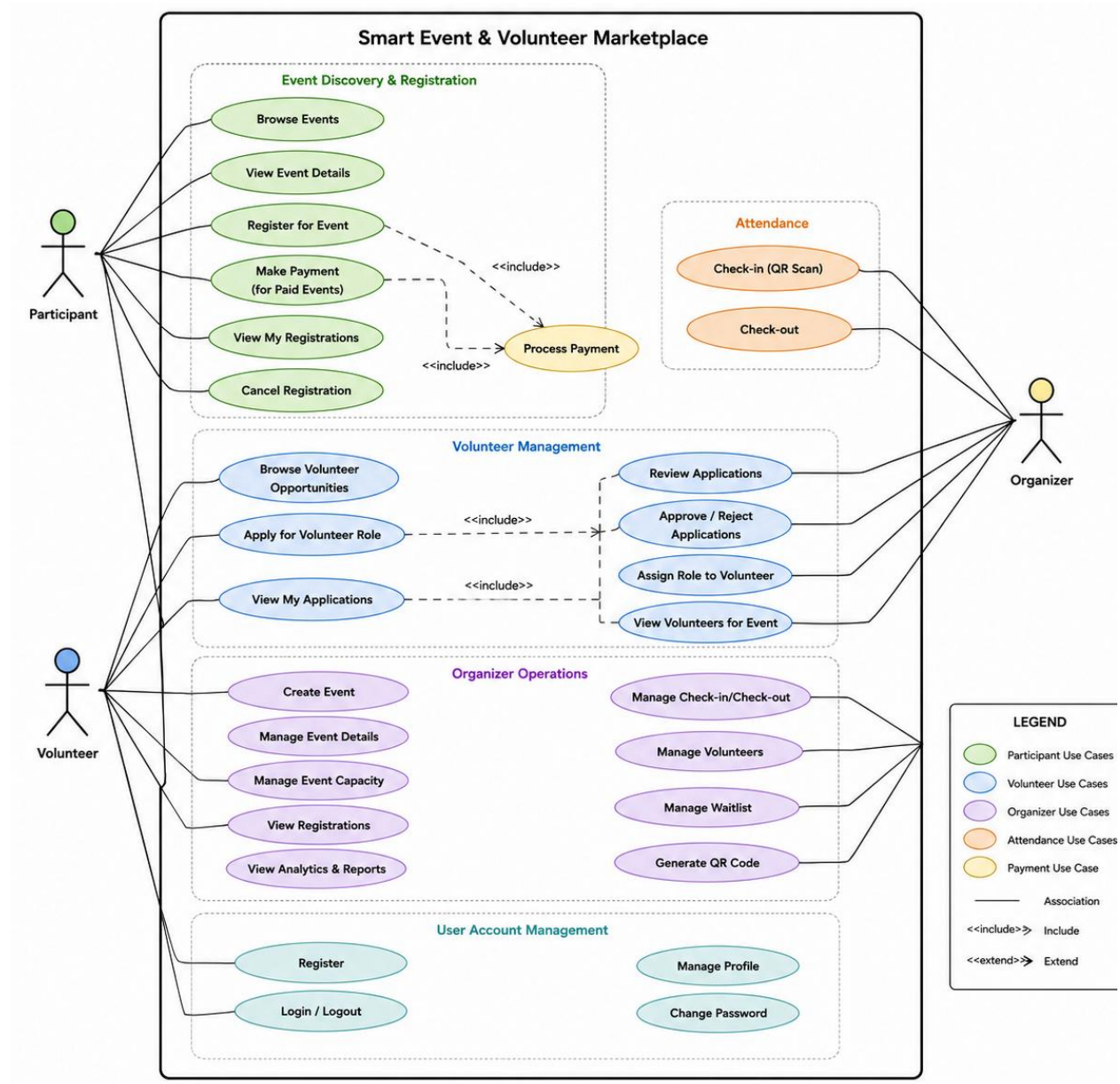
The architecture should support future updates and feature expansion.

3. UML DIAGRAMS

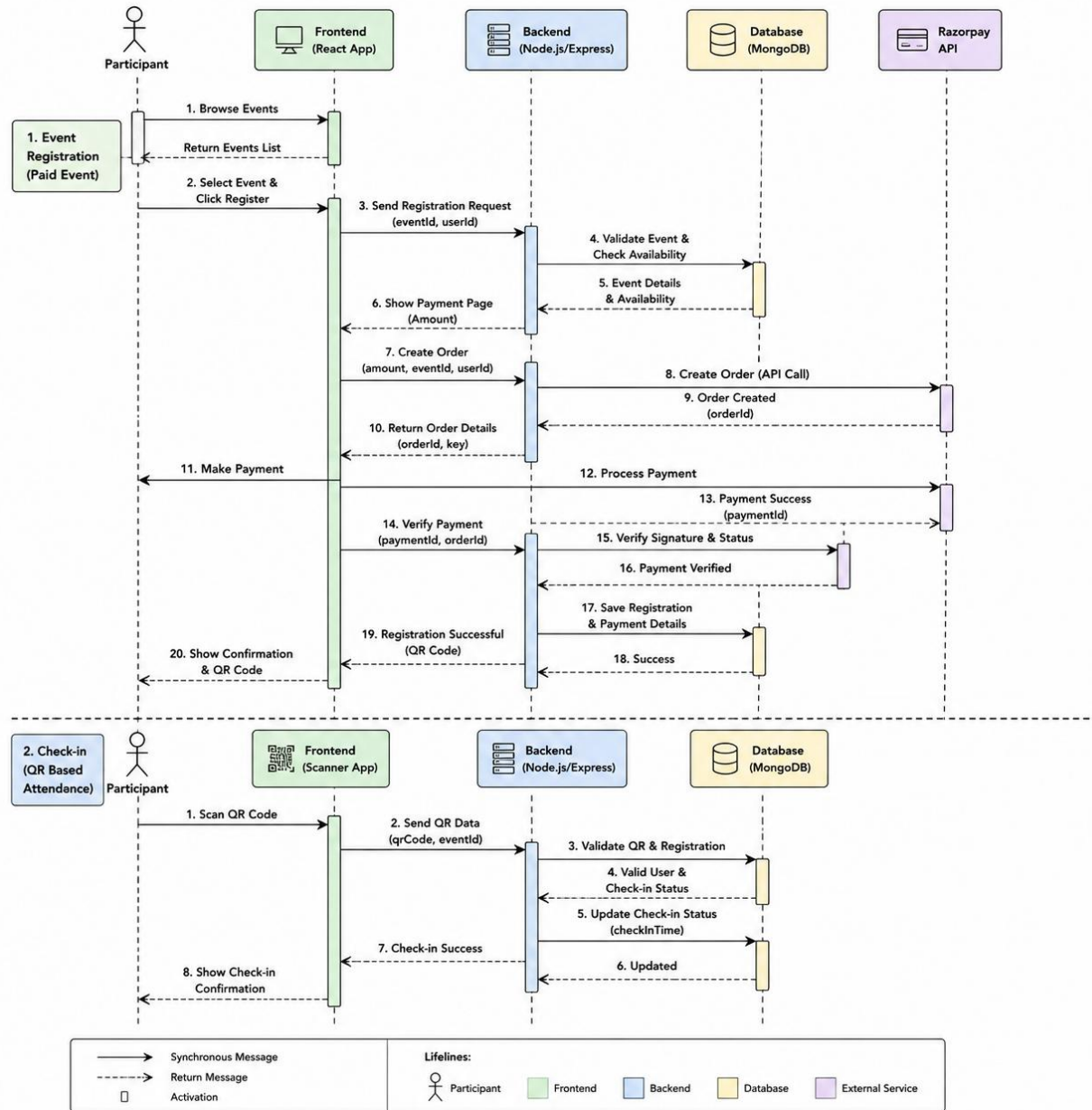
3.1 CLASS DIAGRAM



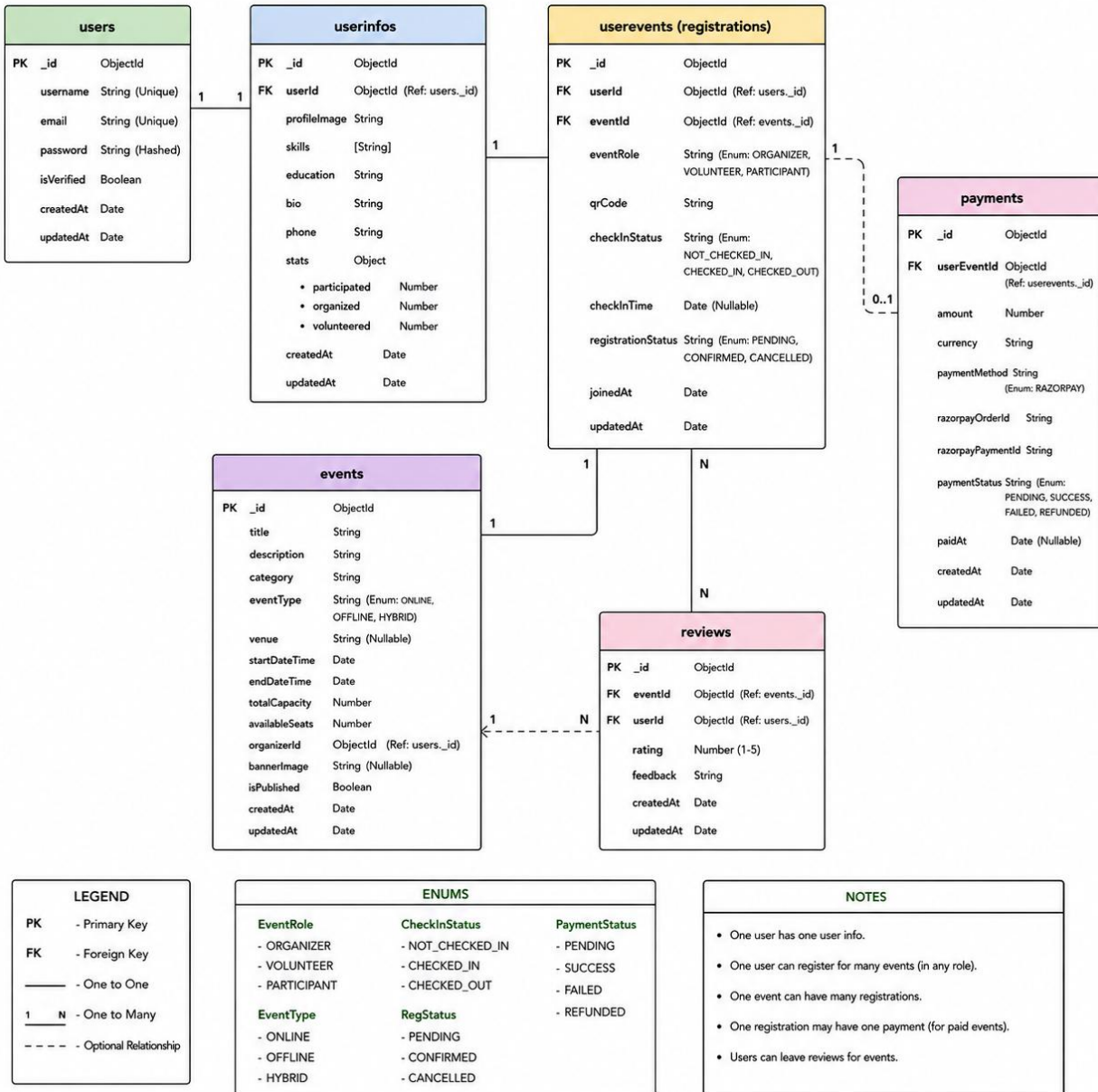
3.2 USE CASE DIAGRAM



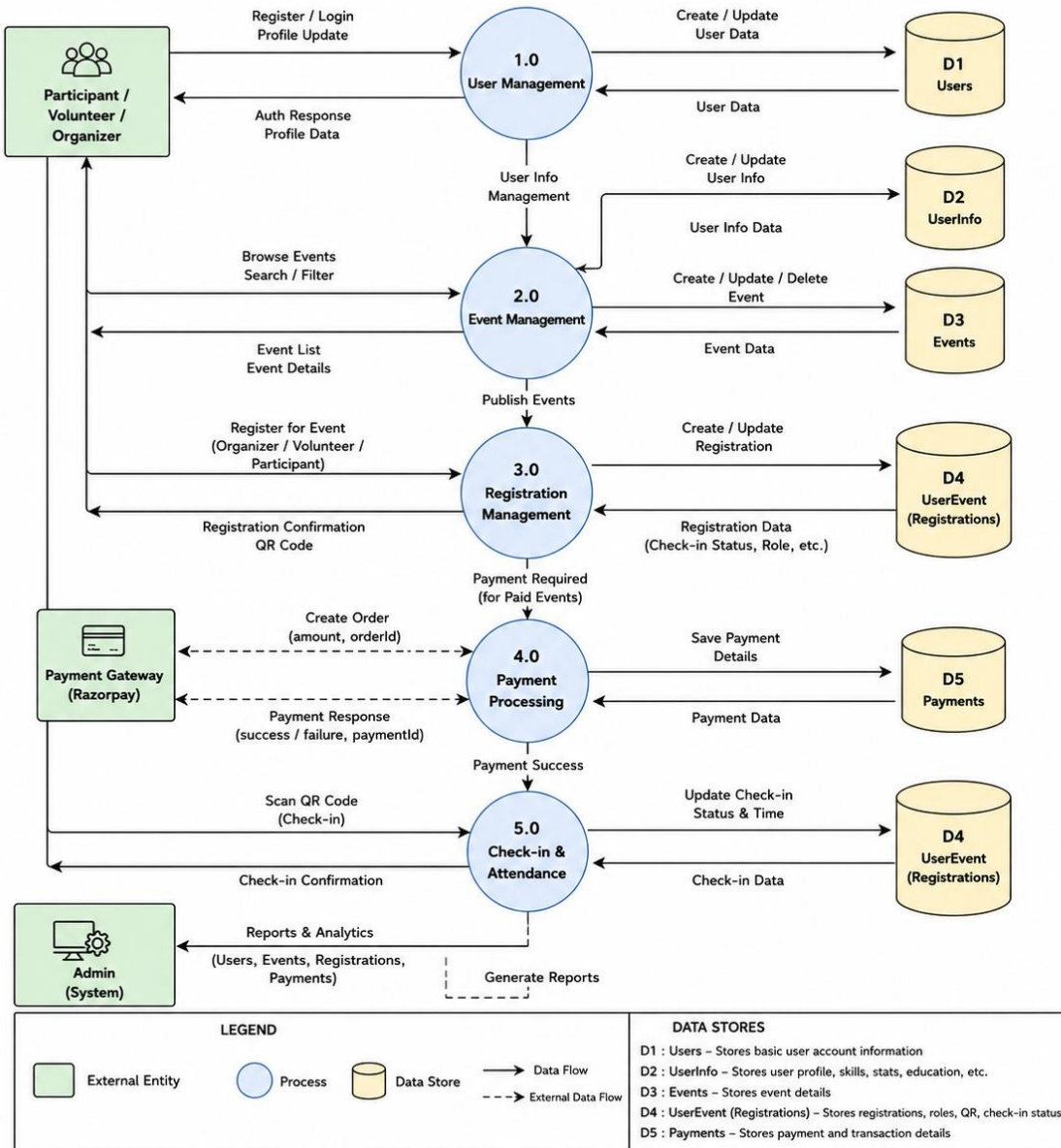
3.3 SEQUENCE DIAGRAM



3.4 SCHEMA



3.5 DATA FLOW DIAGRAM



4. PROJECT DESIGN AND DEVELOPMENT

The project was developed using an iterative software development methodology. The development process was divided into modular phases including requirement analysis, architecture planning, frontend implementation, backend development, integration, testing, and optimization.

The frontend application was designed using React.js to ensure responsive user interaction and efficient state management. Backend APIs were implemented using Express.js and Node.js, while MongoDB handled scalable data storage operations.

Razorpay integration was implemented to support secure payment transactions for paid event registrations. QR generation and scanning modules were added for verified attendance workflows.

The project architecture follows a modular design where each feature operates independently while maintaining centralized communication through APIs and database services.

4.1 Frontend Development

The frontend was developed using React.js with component-based architecture. React Router was used for page navigation, while Axios handled API communication.

The frontend contains:

- Home page
- Event listing pages
- Event details page
- Login and registration pages
- Volunteer application forms
- Organizer dashboard
- QR verification interface

Responsive design principles were implemented to ensure compatibility across desktops, tablets, and smartphones.

4.2 Backend Development

The backend was developed using Node.js and Express.js. REST APIs were created for authentication, event management, volunteer workflows, payment handling, and QR verification.

JWT authentication middleware was implemented to protect private routes and enforce role-based authorization.

Backend modules include:

- Authentication services
- Event management services
- Volunteer management services
- Payment services
- QR verification services
- Analytics services

4.3 Database Design

MongoDB was selected due to its scalability and flexibility in handling event-related data.

Major collections include:

- Users
- Events
- Registrations
- Volunteers
- Payments
- QR Check-ins

Relationships between collections were maintained using ObjectIds and schemas.

4.4 Step-by-Step Development Workflow

The development of the Smart Event & Volunteer Marketplace followed a modular and systematic workflow. The project was implemented in multiple stages beginning with requirement analysis and planning, followed by frontend development, backend implementation, database integration, QR attendance verification, payment integration, and final optimization. This structured approach helped in maintaining clear module separation and simplified testing and integration.

4.4.1 Requirement Analysis and Planning

The initial phase focused on identifying the limitations of existing event management systems and defining the core objectives of the proposed platform. Functional requirements such as event registration, volunteer management, QR attendance verification, organizer dashboards, and payment processing were finalized. The technology stack, database structure, user workflows, and system architecture were also planned during this stage.

4.4.2 Frontend Design and Development

The frontend development process began with designing the user interface using React.js. Core pages such as login, registration, homepage, event listings, event details, volunteer application forms, and organizer dashboards were developed first. React Router was used for navigation, and responsive UI principles were followed to ensure compatibility across multiple devices and screen sizes.

4.4.3 Backend API Development

After the frontend structure, backend services were developed using Node.js and Express.js. REST APIs were implemented for authentication, event management, registrations, volunteer workflows, QR verification, and payment handling. JWT middleware was integrated for secure authentication and role-based authorization.

4.4.4 Database Integration

MongoDB was integrated to store application data including users, user profiles, events, registrations, payments, and attendance records. Mongoose ODM was used to create schemas and maintain relationships between collections for efficient and scalable data management.

4.4.5 QR Verification System

The QR-based attendance verification module was developed after backend integration. Unique QR codes were generated during event registration and linked to registration records. QR scanning functionality was implemented to verify participant and volunteer attendance during event check-ins.

4.4.6 Payment Gateway Integration

Razorpay integration was implemented for paid event registrations. Payment order creation, transaction verification, and payment status updates were added to ensure secure and reliable payment processing within the system.

4.4.7 Testing and Optimization

After integrating all major modules, the application was tested for frontend responsiveness, backend API functionality, QR verification accuracy, and payment workflow reliability. Bugs were fixed and performance optimizations were carried out to improve system stability and overall user experience.

4. TOOLS AND TECHNOLOGIES USED

5.1 Frontend Technologies

React.js

React.js was used for developing the frontend interface of the application. It supports component-based architecture, allowing reusable UI components and efficient state management for event listings, dashboards, registrations, and profile management.

Tailwind CSS / Bootstrap

Tailwind CSS and Bootstrap were used for designing responsive and user-friendly interfaces. These frameworks helped in creating clean layouts and ensuring compatibility across desktops, tablets, and mobile devices.

Axios

Axios was used for handling API communication between the frontend and backend services. It enabled efficient data fetching and request handling for registrations, authentication, payments, and event operations.

5.2 Backend Technologies

Node.js

Node.js was used as the backend runtime environment to handle asynchronous server-side operations and API execution efficiently.

Express.js

Express.js was used for building REST APIs, managing routes, middleware, and implementing backend business logic for event management, registrations, payments, and QR verification.

JWT Authentication

JWT authentication was implemented to provide secure login sessions and role-based access control for participants, volunteers, and organizers.

Razorpay API

Razorpay API was integrated to support secure online payment processing for paid event registrations and transaction verification.

5.3 Database Technologies

MongoDB

MongoDB was used as the primary NoSQL database for storing user details, events, registrations, payments, and attendance records efficiently.

Mongoose ODM

Mongoose ODM was used for defining schemas, validating data, and managing relationships between MongoDB collections.

5.4 Development Tools

Visual Studio Code (VS Code)

VS Code was used as the primary code editor for frontend and backend development due to its extension support and debugging features.

Postman

Postman was used for API testing, request validation, and backend debugging during development.

Git & GitHub

Git and GitHub were used for version control, project backup, and collaborative development.

5.5 Deployment Platforms

Vercel

Vercel was used for deploying and hosting the React.js frontend application with optimized performance and continuous deployment support.

Render / Railway

Render or Railway platforms were used for backend deployment and hosting of Node.js and Express.js services.

6. PROJECT PHASES

6.1. Initiation Phase

- Existing event management systems were analyzed to identify operational limitations and workflow inefficiencies.
- Problems related to volunteer coordination, attendance verification, and registration management were studied.
- The concept of developing a centralized event and volunteer management platform was finalized.
- Core objectives and major functionalities of the proposed system were identified during this phase.

6.2. Planning Phase

- The technology stack including React.js, Node.js, Express.js, MongoDB, and Razorpay was selected.
- System architecture, database structure, and user workflows were designed and documented.
- User roles such as participant, volunteer, and organizer were defined along with their permissions.
- Development modules and implementation sequence were planned before coding began.

6.3. Development Phase

- Frontend pages and reusable UI components were developed using React.js.
- Backend APIs for authentication, event management, registrations, and volunteer workflows were implemented using Express.js and Node.js.
- MongoDB schemas and database relationships were created
- Individual modules were developed and tested incrementally for easier debugging and integration.

6.4. Integration Phase

- Razorpay payment gateway integration was implemented for paid event registrations.
- QR generation and scanning workflows were integrated for attendance verification.
- Frontend components were connected with backend APIs using Axios.
- Organizer dashboards and analytics modules were linked with real-time database operations.

6.5. Optimization Phase

- API response handling and database queries were optimized to improve performance.
- User interface improvements and responsive design refinements were carried out.
- Authentication workflows and error handling mechanisms were improved for better reliability.
- Bugs identified during testing were resolved and system stability was enhanced.

6.6. Deployment Phase

- The frontend application was prepared for deployment using Vercel.
- Backend services and APIs were configured for hosting on Render/Railway platforms.
- Environment variables, database connections, and deployment configurations were finalized.
- Final testing was performed to ensure smooth execution in the production environment.

7. TESTING

This section describes the testing methodologies followed to ensure that the Smart Event & Volunteer Marketplace platform delivers a reliable, secure, and efficient experience for participants, volunteers, and organizers. Different testing methods were performed to validate system functionality, module integration, payment workflows, QR verification, and overall platform performance.

7.1 Unit Testing

Unit testing was conducted to verify the functionality of individual modules and components independently.

Approach:

- **Component Isolation:** Tested modules such as authentication, event registration, QR generation, and payment verification individually.
- **API Validation:** Verified backend API responses and request handling for different user actions.
- **Input Validation:** Checked form validations, error handling, and field constraints during registrations and login operations.

7.2 Integration Testing

Integration testing ensured smooth interaction between frontend components, backend APIs, database services, and third-party integrations.

Approach:

- Cross-Module Testing: Validated workflows involving registrations, volunteer applications, payments, and QR verification.
 - Database Synchronization: Tested data flow between frontend interfaces and MongoDB collections.
 - Third-Party Integration: Verified communication with Razorpay payment services and QR generation modules.
-

7.3 Functional Testing

Functional testing ensured that all core features of the platform worked according to the specified requirements.

Approach:

- End-to-End Testing: Tested complete workflows such as event registration, volunteer approval, payment completion, and QR check-ins.
- Role-Based Testing: Verified functionality for participants, volunteers, and organizers separately.
- Edge Case Handling: Tested invalid inputs, duplicate registrations, expired QR scans, and payment failures.

7.4 Usability Testing

Usability testing focused on evaluating the user interface, navigation flow, and overall ease of use of the platform.

Approach:

- User Feedback: Conducted testing sessions with students and event organizers to observe real usage behavior.
- Navigation Analysis: Evaluated accessibility of major features such as event browsing, registrations, and dashboards.
- UI Improvements: Collected suggestions to improve responsiveness, layout consistency, and user interaction.

7.5 Performance Testing

Performance testing was performed to measure system responsiveness, API efficiency, and reliability under different workloads.

Approach:

- Load Testing: Simulated multiple concurrent event registrations and payment requests.
- API Performance: Tested backend response times for registrations, QR verification, and dashboard analytics.
- Device Compatibility: Verified frontend responsiveness across desktops, tablets, and mobile devices.

7.6 Regression Testing

Regression testing ensured that newly added features and updates did not affect existing functionalities.

Approach:

- Re-Testing: Existing modules were re-tested after implementing new features or fixing bugs.
- Workflow Validation: Checked authentication, registrations, payments, and QR verification after backend updates.
- Stability Testing: Ensured system reliability after integrating Razorpay and organizer analytics modules.

7.7 User Acceptance Testing (UAT)

User Acceptance Testing was conducted to verify that the platform met real-world user expectations and operational requirements.

Approach:

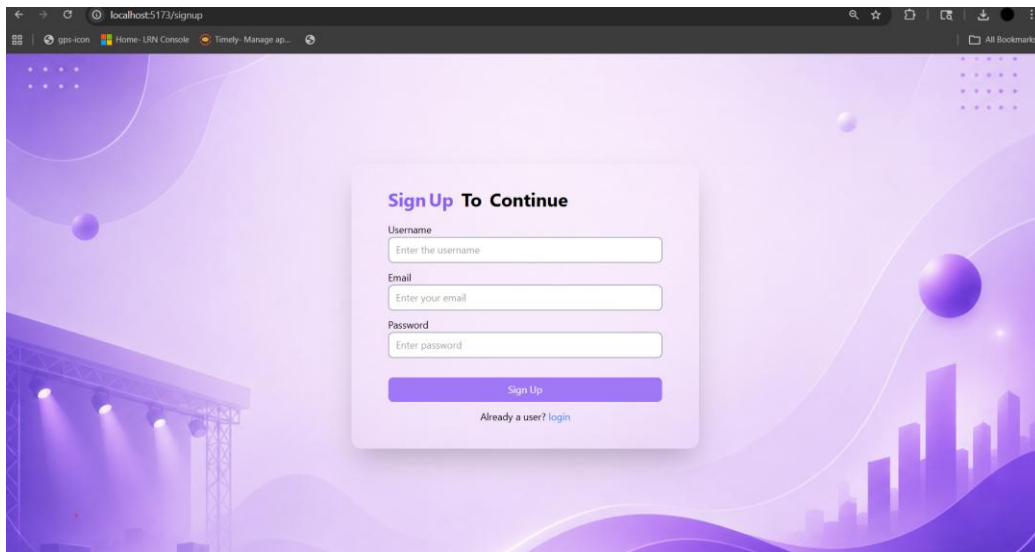
- Beta Testing: Trial usage was conducted with students, volunteers, and organizers.
- Live Workflow Observation: Users performed event registrations, volunteer applications, payments, and QR check-ins.
- Feedback Collection: Suggestions and observations were collected to improve usability, workflow efficiency, and overall platform experience.

8. USER GUIDE

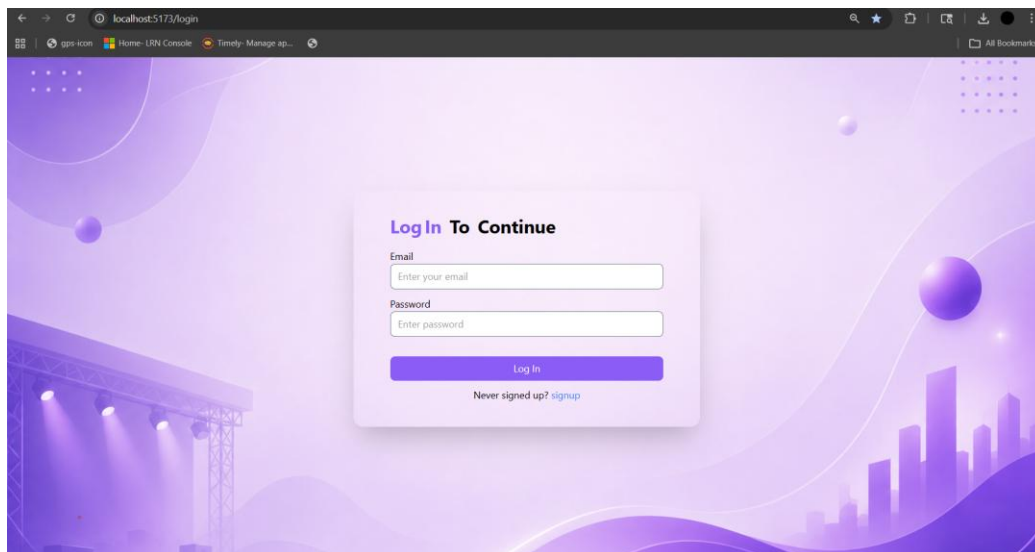
This User Guide walks users through the important pages of the platform and the main actions of the application.

8.1. Authentication Pages

These are the pages that allow the users to securely register/login to the platform



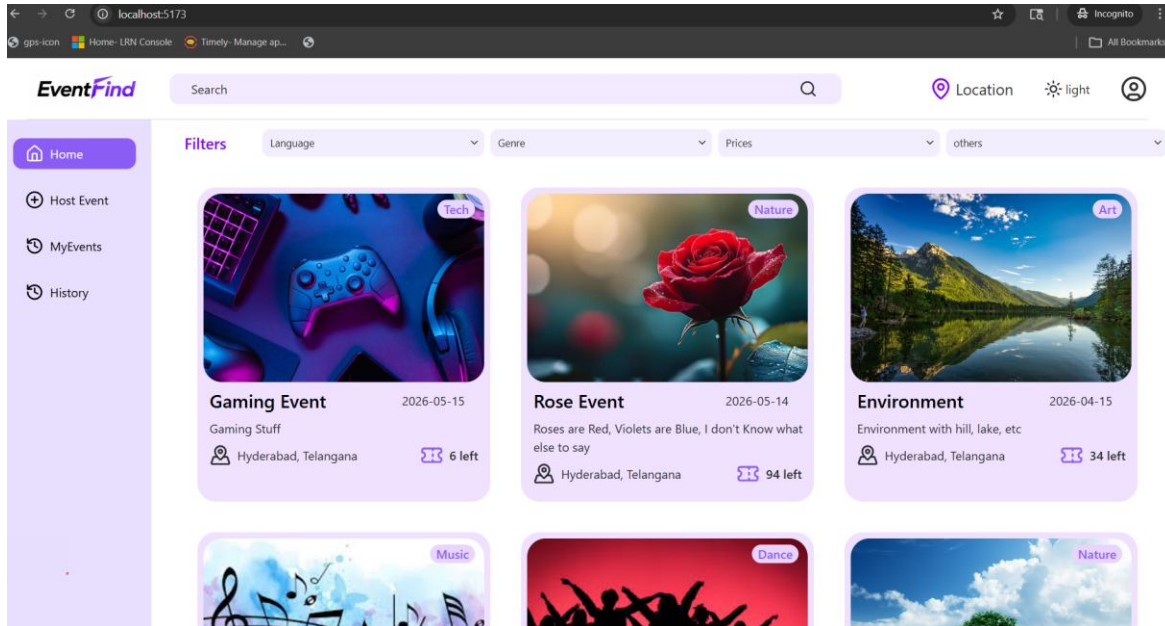
A screenshot of a web browser displaying the 'Sign Up' page. The browser's address bar shows 'localhost:5173/signup'. The page has a purple-themed background with abstract shapes and a city skyline. In the center, there is a white card with the title 'Sign Up To Continue'. Below the title are three input fields: 'Username' (with placeholder text 'Enter the username'), 'Email' (with placeholder text 'Enter your email'), and 'Password' (with placeholder text 'Enter password'). A purple 'Sign Up' button is at the bottom of the card, with a link 'Already a user? login' below it.



A screenshot of a web browser displaying the 'Log In' page. The browser's address bar shows 'localhost:5173/login'. The page has the same purple-themed background as the Sign Up page. In the center, there is a white card with the title 'Log In To Continue'. Below the title are two input fields: 'Email' (with placeholder text 'Enter your email') and 'Password' (with placeholder text 'Enter password'). A purple 'Log In' button is at the bottom of the card, with a link 'Never signed up? signup' below it.

8.2. Home Page

This is the page where the users are listed. Users browse events using filters and search features.



8.3. Event Page:

Users browse event details and can register and apply as volunteer from this page



LOCATION
JN Auditorium, JNTUH, Hyderebad

CATEGORY
Tech

About this Event

Hands-on workshop on Machine Learning and Generative AI

Terms & Conditions

- Valid college ID is mandatory for entry.
- Registrations are non-transferable.
- Event details may change if required.
- Misconduct may result in removal from the event.

Organised By

spoodyy
spoodyy@123
★ Verified Organizer

AI Workshop

250 seats left 0 joined

2026-05-20

10:00

6 hours

English

JN Auditorium, JNTUH, Hyderebad

Tech

PRICE
₹ **99**

Apply as Volunteer

Book Now

8.4. Profile Page:

Users can create and update their profiles. Users can see the profiles of the organizers, volunteers and participants.

Ravi
ravi96@gmail.com

ABOUT

2nd year CSE student at JNTUH looking forward to participate in technical events and hackathons

ACTIVITY

1
Participated

0
Organized

0
Volunteered

SKILLS

Problem Solving
Full Stack Development
C++

EDUCATION

JNTUH UCESTH

SKILLS

Problem Solving
Full Stack Development
C++

EDUCATION

JNTUH UCESTH

EVENT HISTORY

Participated
Organized
Volunteered

Tech Meetup 2026
2026-05-20

8.5. Organizer Dashboard

Organizers monitor event operations, registrations, volunteer activity, and analytics.

The screenshot shows the 'Host Event Page' for 'Tech Meetup 2026'. The dashboard includes a sidebar with navigation links: Home, Host Event, MyEvents, and History. The main content area features three summary cards: 'Registrations / 200' with a value of 1, 'Total Revenue' of ₹ 100, and 'Checked In' with a value of 0. Below these is a section titled 'See your live event page' with buttons for 'Myevent Page' and 'Scan QR'. A tabbed interface shows 'Booking Details' and 'Volunteer Applications'. The 'Booking Details' tab displays a table with the following data:

Registration ID	Username	Email	Registered At	Status	Checked In	Checked In At
3014ac8b	Ravi	ravi96@gmail.com	2026-05-11, 17:11:50	registered	No	Not scanned

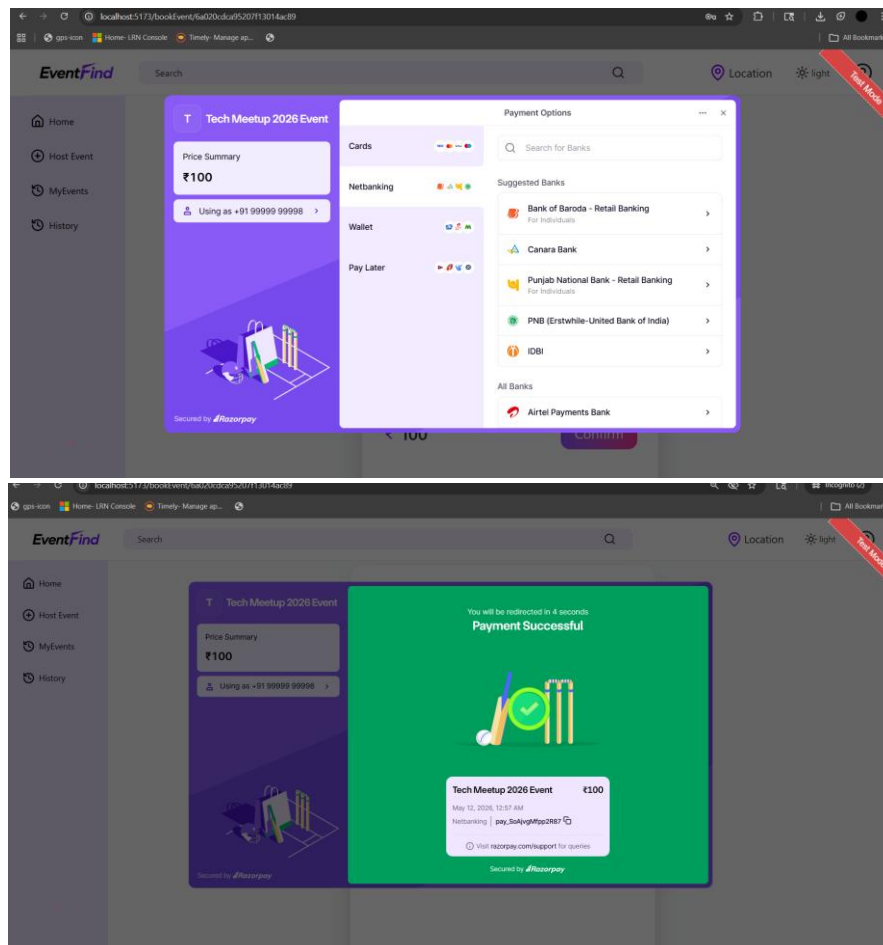
8.6. Payments

Paid event registrations are completed through Razorpay checkout.

The screenshot shows the 'Confirm Your Registration' page for 'Tech Meetup 2026'. The page displays event details in a grid format:

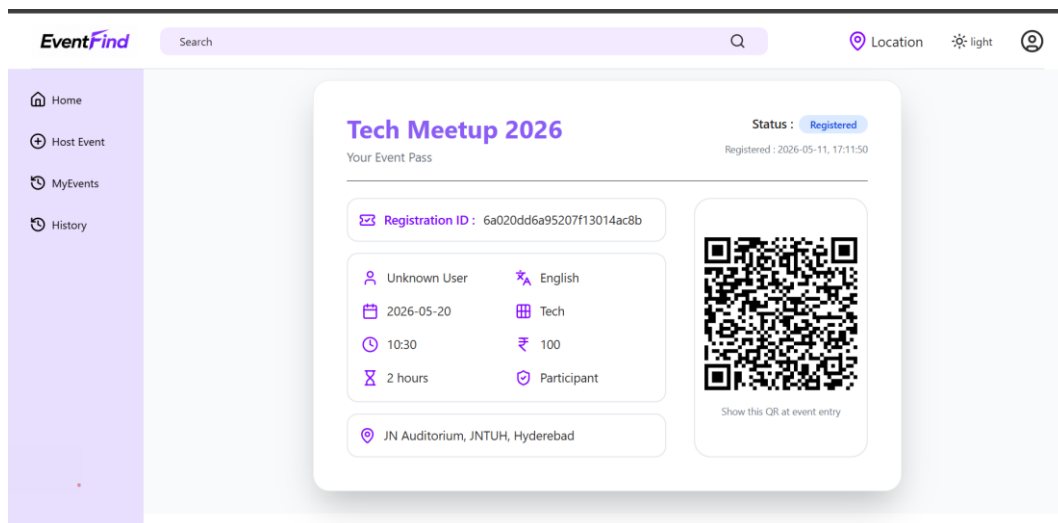
- Date: 2026-05-20
- Category: Tech
- Time: 10:30
- Slots: 200
- Duration: 2 hours
- Price: ₹ 100
- Language: English
- Location: JN Auditorium, JNTUH, Hyderebad

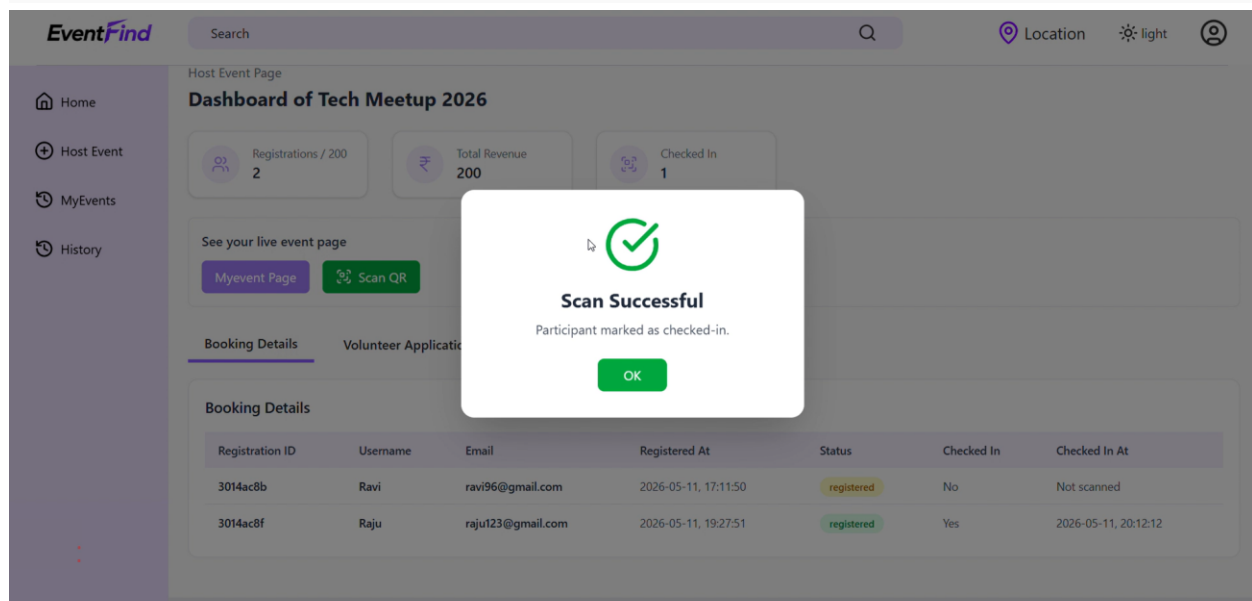
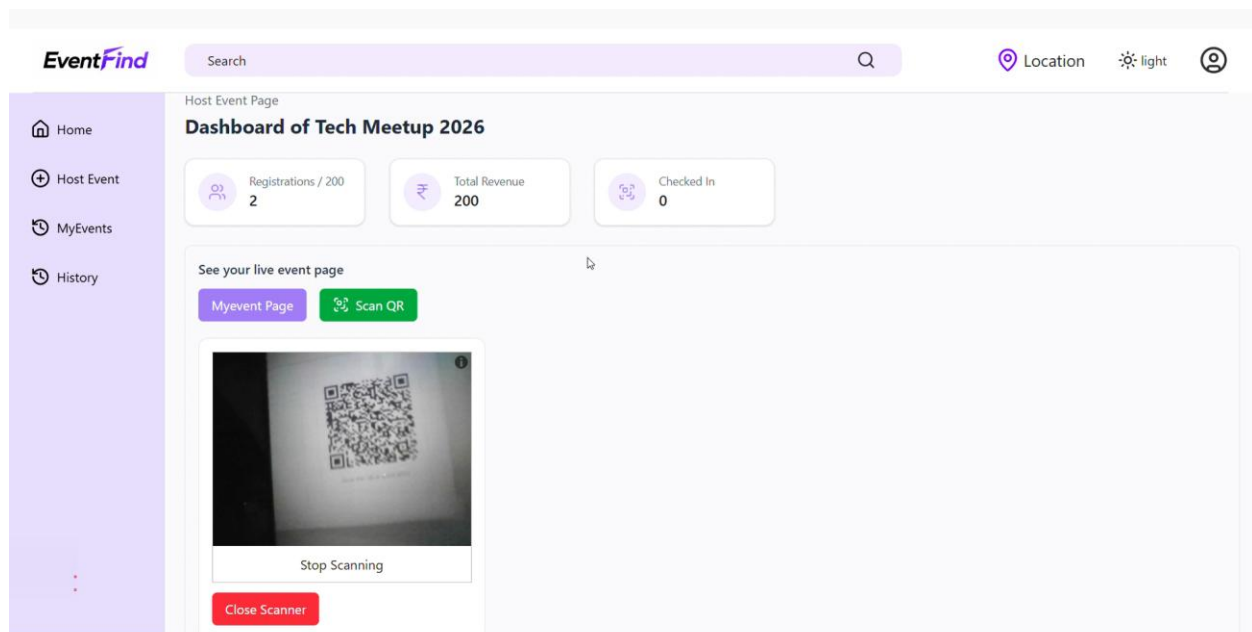
At the bottom, there is a summary showing '₹ 100' and a 'Confirm' button.



8.7. QR Verification

Participants scan QR codes during check-in for attendance confirmation.



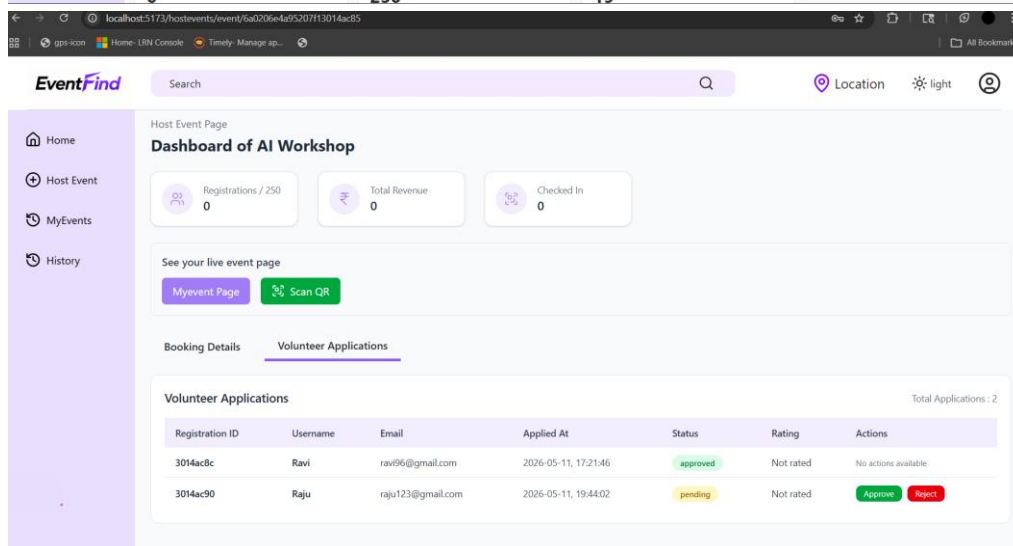


8.8. Volunteer Workflow

Users apply for volunteer roles and track approval status. Organizers can either approve or reject that applications



The top screenshot shows the EventFind AI Workshop page. The main banner features the text "ARTIFICIAL INTELLIGENCE WORKSHOP" with a "REGISTER NOW" button. The sidebar on the right lists event details: 250 seats left, 0 joined, date 2026-05-20, time 10:00, duration 6 hours, language English, location JN Auditorium, JNTUH, Hyderabad, category Tech, and price ₹ 99. A "Waiting for approval" status is shown with a "Withdraw Application" button.

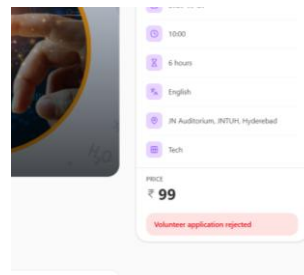
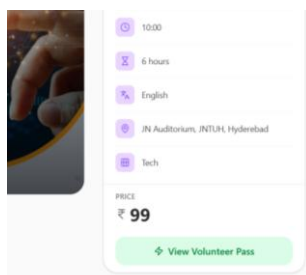


The bottom screenshot shows the "Dashboard of AI Workshop" for the host. It displays registration statistics: Registrations / 250 (0), Total Revenue (0), and Checked In (0). Below this, there are buttons for "Myevent Page" and "Scan QR". The "Volunteer Applications" section shows a table with 2 total applications.

Registration ID	Username	Email	Applied At	Status	Rating	Actions
3014ac8c	Ravi	ravi96@gmail.com	2026-05-11, 17:21:46	approved	Not rated	No actions available
3014ac90	Raju	raju123@gmail.com	2026-05-11, 19:44:02	pending	Not rated	<button>Approve</button> <button>Reject</button>

Application Approved

Application Rejected



9. CONCLUSION

The Smart Event & Volunteer Marketplace successfully addresses major limitations in traditional event management systems by integrating event registration, volunteer coordination, attendance verification, and organizer analytics into a centralized platform.

The project demonstrates the practical implementation of modern MERN-stack technologies along with secure payment processing and QR-based participation verification. The platform improves operational transparency, reduces manual workload, and enhances participant experience.

Future enhancements may include AI-driven recommendations, mobile applications, automated certificates, and deeper analytics integrations. The modular architecture ensures scalability and long-term adaptability for institutional deployment.

10. REFERENCES

1. React Documentation – <https://react.dev/>
2. Node.js Documentation – <https://nodejs.org/en/docs>
3. Express.js Documentation – <https://expressjs.com/>
4. MongoDB Documentation – <https://www.mongodb.com/docs/>
5. Razorpay API Documentation – <https://Razorpay.com/docs>
6. JWT Authentication – <https://jwt.io/introduction>
7. MDN Web Docs – <https://developer.mozilla.org/>
8. GitHub Documentation – <https://docs.github.com/>